

CM12002

Computer Systems

Architectures

Boolean Logic and Circuits

Fabio Nemetz

Simplification

We will develop simplification as a two stage process :

1. First convert the Boolean formula to a ***standard sum of products***.
2. Then find the simplest form for this formula (the one with the fewest logic gates).

Karnaugh maps

For any formula, we can find the simplest equivalent **sum-of-products** expression using **Karnaugh maps**.

(also called [Veitch diagrams](#), [Karnaugh/Veitch diagrams](#), [Carroll diagrams](#)).

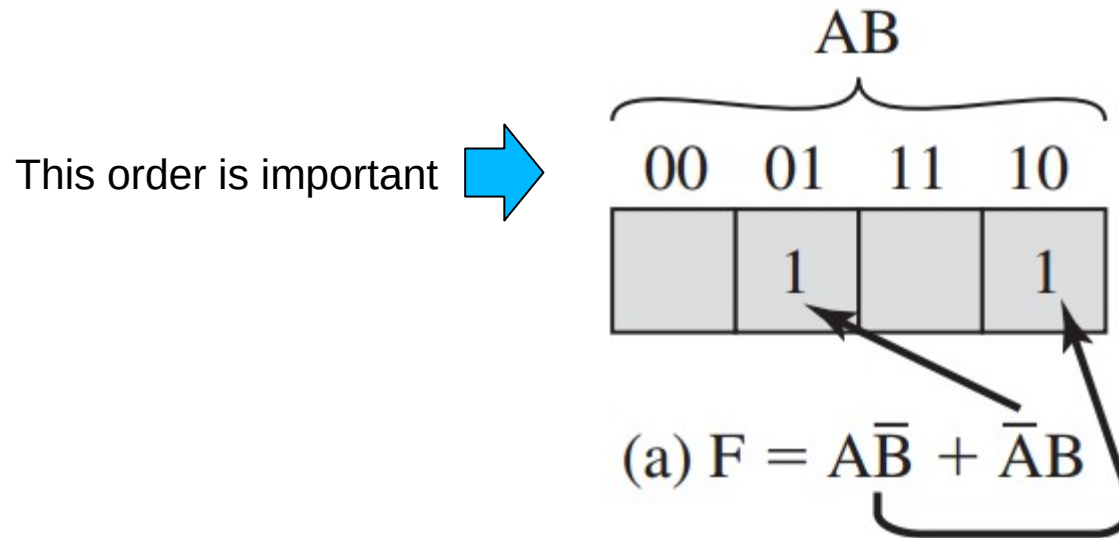
Karnaugh maps:

- a method to simplify boolean algebra expressions
- reduces the need for extensive calculations by taking advantage of humans' pattern-recognition capability.
- works well for up to 4 variables

Karnaugh maps

Any function of **two Boolean variables**, A and B, can be plotted on a map with **four regions**:

example: $F = AB' + A'B$

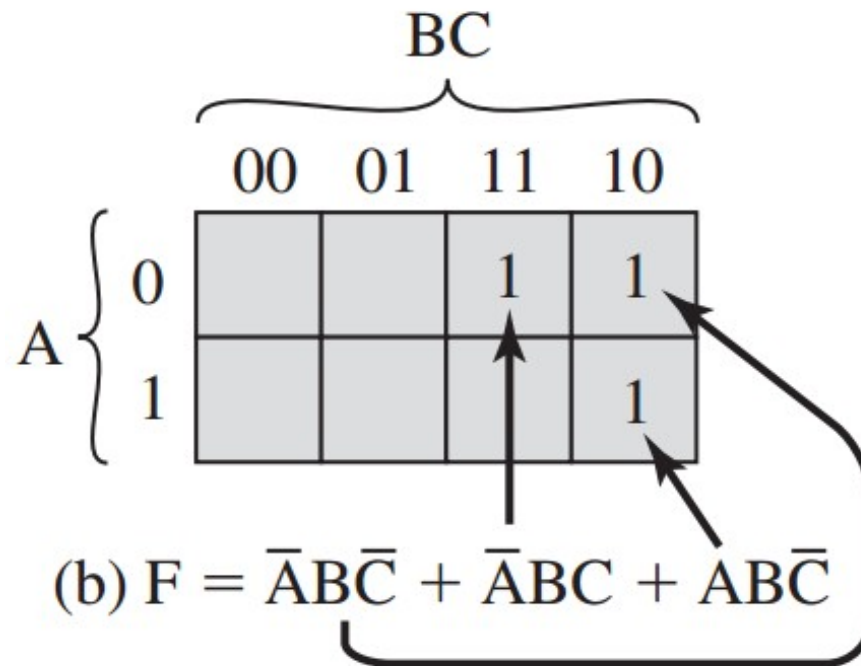


Notice that any function of the two variables in the form of a standard sum of products can be plotted directly into the map.

Karnaugh maps

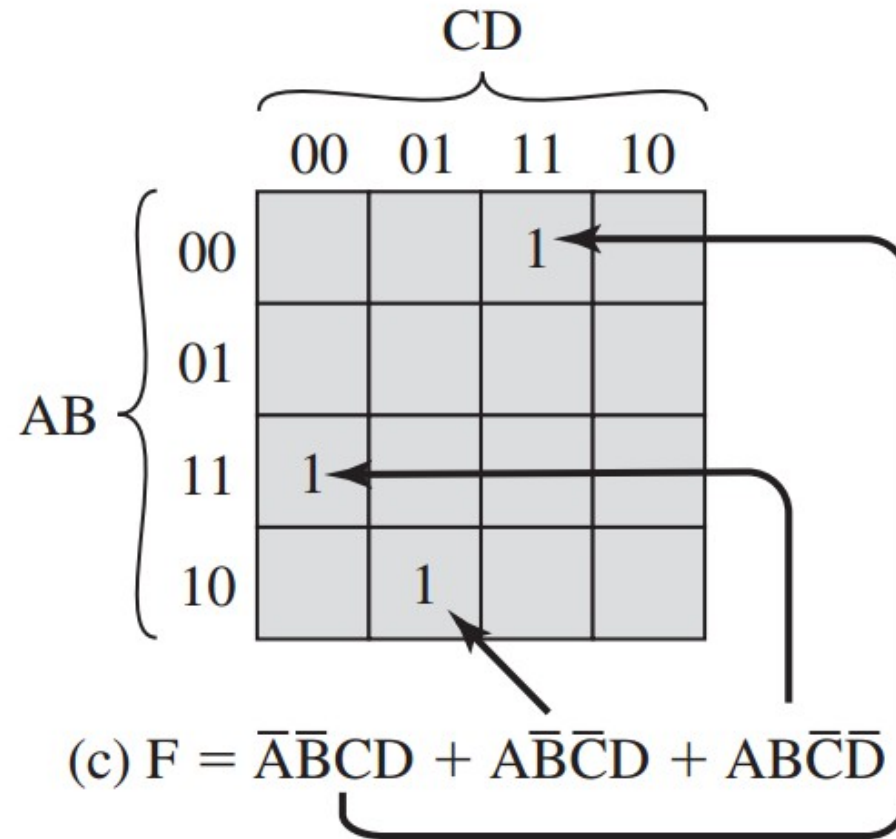
With **three variables** - A, B and C - in the Boolean function, a map with **eight regions** will be needed:

example: $F = A'BC' + A'BC + ABC'$



Karnaugh maps

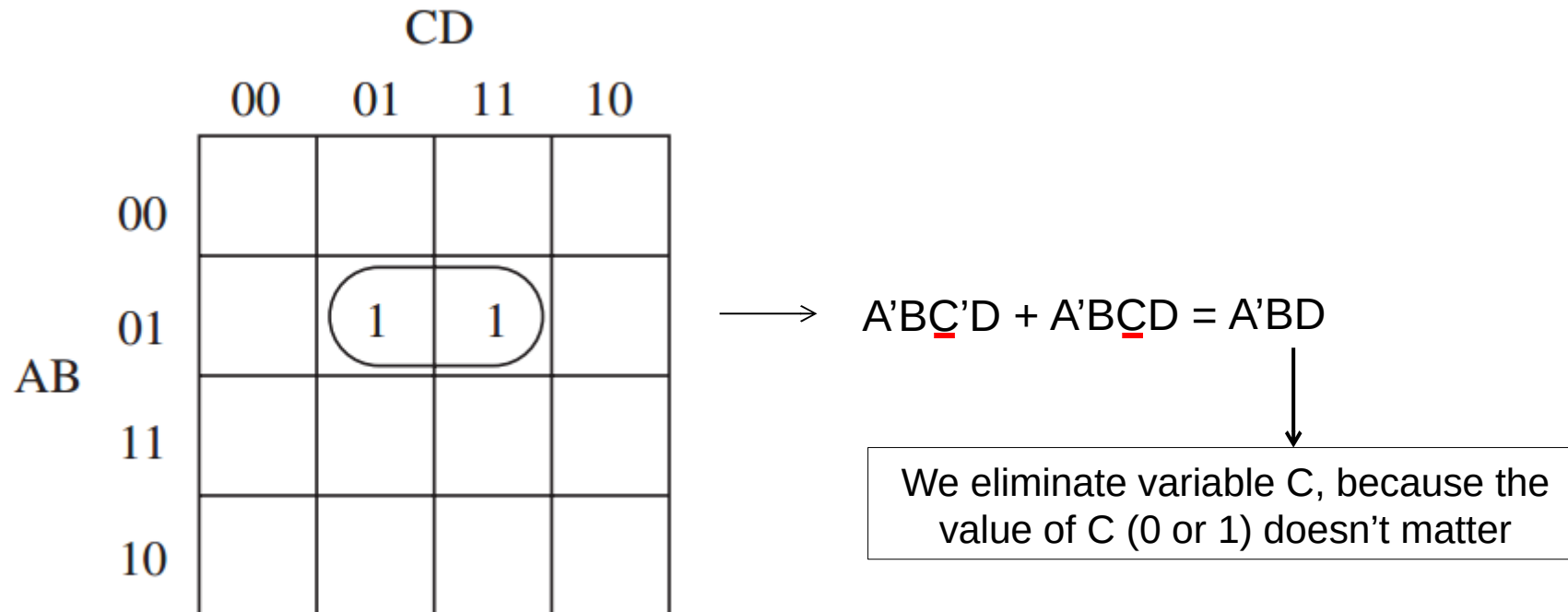
With **four variables** - A, B, C and D - in the Boolean function, a map with **sixteen regions** will be needed:



Karnaugh maps

Once map is created, note the arrangement of 1s on the map

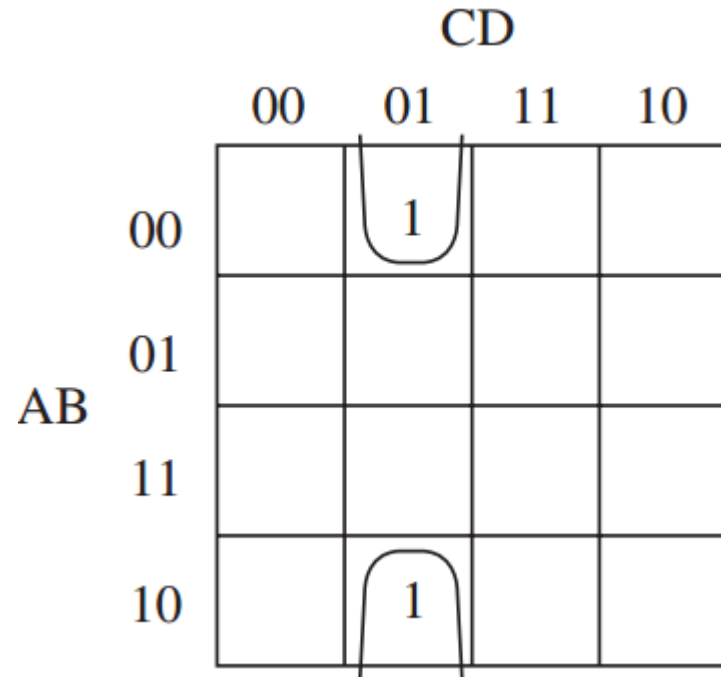
- Any two adjacent squares with entries of 1 → corresponding product terms differ in only one variable:
 - This means we can eliminate one variable



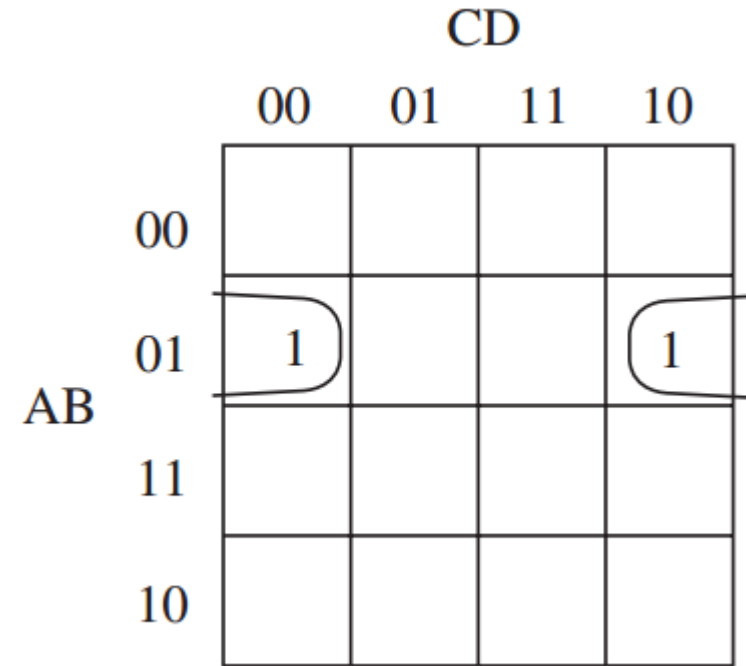
Note: this is not the same example from the previous slide

Karnaugh maps

Adjacency can be extended to include wrapping around the edge



$$A'B'C'D + AB'C'D = B'C'D$$



$$A'BC'D' + A'BCD' = A'BD'$$

Karnaugh maps

Adjacent squares can be grouped not only in 2s, but also in 4s, 8s, ...
(more variables can be eliminated)

		CD			
		00	01	11	10
AB	00	1	1	1	1
	01				
	11				
	10				

$$A'B'C'D' + A'B'C'D + A'B'CD + A'B'CD' =$$

$$A'B'$$

		CD			
		00	01	11	10
AB	00				
	01	1	1		
	11	1	1		
	10				

$$A'BC'D' + A'BC'D + ABC'D' + ABC'D =$$

$$BC'$$

		CD			
		00	01	11	10
AB	00				
	01	1			1
	11	1			1
	10				

$$A'BC'D' + A'BCD' + ABC'D' + ABCD' =$$

$$BD'$$

Karnaugh maps

Examples of groups of 8 adjacent squares

		CD			
		00	01	11	10
AB	00	1	1	1	1
	01	1	1	1	1
	11				
	10				

A'

		CD			
		00	01	11	10
AB	00	1			1
	01	1			1
	11	1			1
	10	1			1

D'

		CD			
		00	01	11	10
AB	00			1	1
	01			1	1
	11			1	1
	10			1	1

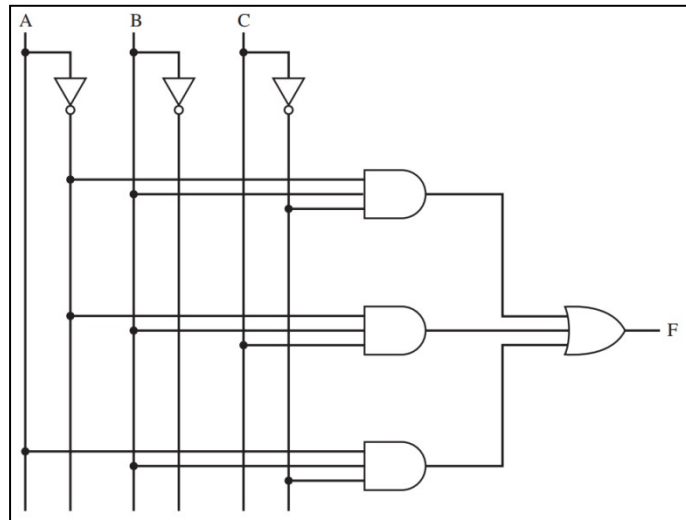
C

Example

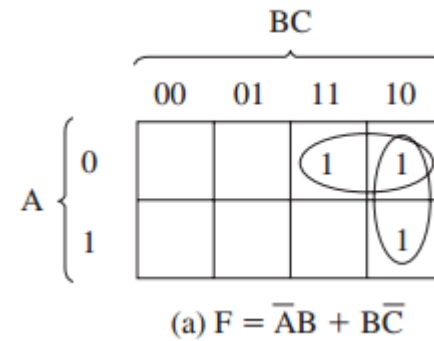
$$F = \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$



Direct implementation



Karnaugh map



In this case, we'll need only 5 gates instead of 7

However it is possible to simplify it further. Let's see a general procedure – we'll get back to this example later

Simplifying Boolean expressions

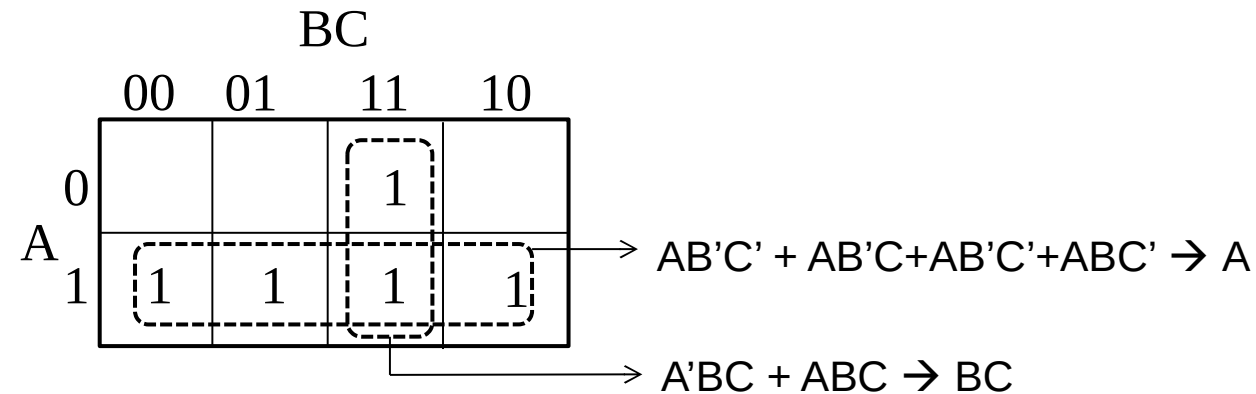
- 1) Write it in the form of a *standard sum of products*;
- 2) Plot the product terms on a *Karnaugh map*;
- 3) Find the *smallest number* of the *biggest valid groupings* that will encompass *all of the marked squares (1s)* of the Karnaugh map; and
- 4) *Read off the function* that results.

Karnaugh maps

We can try this out on our function $F = A + B.C$ (previous lecture)

1) We know that the **standard sum of products** form of this function is:
$$F = A.B.C + A.B.C' + A.B'.C + A.B'.C' + A'.B.C$$

2) The Karnaugh map is:



3) The two areas give us: $F = A + B.C$

Karnaugh maps

The result of this sequence of operations shows us that

$$F = A + B.C$$

is indeed the *simplest form* of this Boolean function.

But what do we mean by simplest?

If the answer is: requiring the fewest logic gates to implement, then what we get from the Karnaugh map is the simplest (non-standard) sum of products representation.

We may need to do a little further simplification.

For example, the form

$$M = A.B + A.C$$

which requires two AND gates and one OR gate, can be factorised into

$$M = A.(B + C)$$

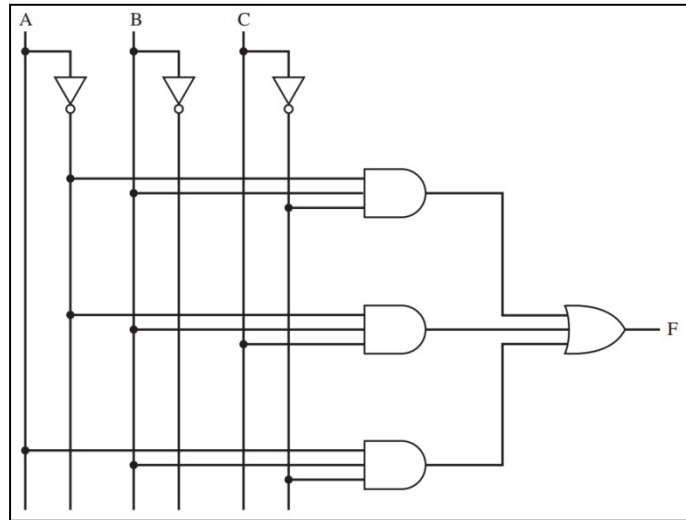
which needs only one AND gate and one OR gate.

From the previous example

$$F = \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$



Direct implementation

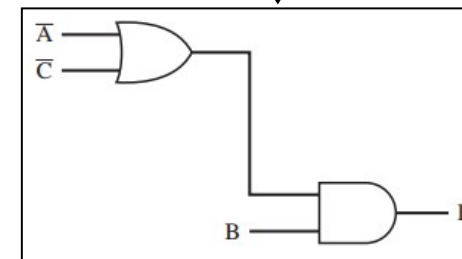


Karnaugh map

		BC			
		00	01	11	10
A	0			1	1
	1				1

(a) $F = \overline{A}B + B\overline{C}$

Further simplification
 $= B(A' + C')$



Tools

[Home](#) [2_variables](#) [3_variables](#) [4_variables](#) [5_variables](#) [6_variables](#) [7_variables](#) [8_variables](#)

[Input interface](#) [Truth Table](#) [Kmap with Don't cares \(SOP\)](#) [Kmap without Don't cares \(SOP\)](#)

Truth Table

Submit		Y						
	A	B	C	D	0	1	x	
0	0	0	0	0	☒	☐	☐	
1	0	0	0	1	☒	☐	☐	
2	0	0	1	0	☒	☐	☐	
3	0	0	1	1	☒	☐	☐	
4	0	1	0	0	☒	☐	☐	
5	0	1	0	1	☒	☐	☐	
6	0	1	1	0	☒	☐	☐	
7	0	1	1	1	☒	☐	☐	
8	1	0	0	0	☒	☐	☐	
9	1	0	0	1	☒	☐	☐	
10	1	0	1	0	☒	☐	☐	
11	1	0	1	1	☒	☐	☐	
12	1	1	0	0	☒	☐	☐	
13	1	1	0	1	☒	☐	☐	
14	1	1	1	0	☒	☐	☐	
15	1	1	1	1	☒	☐	☐	

☐ Highlight groups (SOP Kmap)
☒ SOP
☐ POS
☐ Quine-McCluskey Method (SOP)

☒ CSS circuit drawing
☐ ASCII circuit drawing

Try Tab and arrow keys for keyboard input.

<http://www.32x8.com/var4.html>

Conclusions

In this lecture, we have

- Shown how to simplify formulas in sum-of-products form using **Karnaugh maps**.